TECHNICAL REPORT

Virtual Swarm Drone Coordination

A Comprehensive Study on Autonomous Multi-Agent Monitoring Systems

Sayon Manna | M.Srijaswini | V.Akhil krishna | B.Tech

Guide 1 - Dr. Shivashankar

Guide 2 - Dr. Rohini B S

Jain (Deemed-to-be University), Bengaluru, Karnataka, India

May 2026

1. Main Goal, Agenda, and Necessity for Monitoring

1.1 What Is This System?

Virtual Swarm Drone Coordination is a simulation platform designed to model how a group of autonomous drone agents can be coordinated, monitored, and directed toward mission objectives without any single controller managing every movement. The system treats each drone as an independent agent that perceives its local environment, reports its status, and reacts to high-level commands issued by a designated leader. The leader, in turn, draws on either a large language model or a deterministic rule engine to decide what the swarm should prioritize next.

The platform is built in React and TypeScript, runs entirely in the browser, and is designed to be experimented with in real time. It is not a toy animation — it implements genuine coordination semantics, role differentiation, formation stability mechanisms, hazard response, and AI-assisted mission planning.

1.2 The Core Agenda

The agenda of this system centers on one fundamental question: can a group of simple agents, each acting only on local information, produce globally useful and stable collective behavior when guided by a lightweight AI brain?

This question matters because real-world drone deployments — in disaster relief, border surveillance, environmental monitoring, search and rescue, and infrastructure inspection — all demand exactly this kind of coordination. A single operator cannot manually control fifty drones at once. An autonomous fleet must self-organize, detect problems internally, report threats to a commander, and adjust behavior without moment-to-moment human intervention.

- Demonstrate that decentralized local rules can produce emergent, globally stable formations and behaviors
- Show that a language model can function as a strategic mission planner without needing to control every drone movement
- Prove that the system remains operational when the AI is unavailable, through a deterministic fallback
- Provide a testbed for experimenting with different formation modes, hazard scenarios, and drone role configurations
- Establish an architecture that scales cleanly from small fleets to larger deployments without redesigning the coordination logic

1.3 Why Monitoring Requires This Approach

Conventional monitoring solutions rely on a central server that receives all data, processes it, and dispatches commands. This works acceptably for small deployments but breaks down at scale for several reasons. Centralized systems have a single point of failure. Communication latency between drones and the central server compounds as the fleet grows. And a central system that knows everything must also process everything — which is computationally expensive.

A swarm-based monitoring architecture solves these problems by distributing perception, computation, and decision-making across the fleet itself. Each drone only needs to know what is immediately around it. The leader only needs a compressed summary of the swarm state, not raw sensor feeds from every agent. This means the system degrades gracefully under communication failure rather than collapsing entirely.

Key Monitoring Advantages of Swarm Architecture

- Redundancy: losing one drone does not break the mission — others adapt
- Scalability: coordination cost grows linearly, not quadratically, with fleet size
- Resilience: the local fallback planner keeps the swarm functioning when AI is offline
- Coverage: distributed agents can monitor a far larger area than any single drone
- Real-time adaptation: the swarm reacts to new hazards within the same simulation tick

2. Literature Review

2.1 Foundations of Swarm Intelligence

The field of swarm intelligence traces its formal origins to the work of Dorigo and Colorni in the early 1990s with ant colony optimization, and to Craig Reynolds whose 1987 SIGGRAPH paper introduced the Boids model. Reynolds showed that three simple local rules — separation, alignment, and cohesion — applied independently by each agent produce convincing collective motion without any central director. This insight became the cornerstone of virtually all subsequent work on flocking simulation and distributed multi-agent coordination.

Bonabeau, Dorigo, and Theraulaz consolidated the theoretical foundations of swarm intelligence in their 1999 Oxford University Press volume, establishing the key properties that make swarm systems useful: robustness, flexibility, self-organization, and scalability. These four properties remain the design targets that the current simulation attempts to demonstrate in a browser-accessible environment.

2.2 Formation Control Research

Formation control literature has developed along three main threads. Leader-follower approaches, introduced by Desai, Ostrowski, and Kumar in 2001, assign one robot to lead while others maintain geometric offsets from it. This is simple to implement but creates fragility — if the leader fails, the formation collapses.

Virtual structure approaches, developed by Lewis and Tan in 1997, treat the formation as a rigid body and assign each robot to a position within it. This produces tighter formations but struggles with dynamic reconfiguration. Behavior-based formation control, introduced by Balch and Arkin in 1998, uses local steering behaviors to maintain formation positions without explicit inter-robot communication, making it more robust to individual failures.

The current simulation draws most directly from the behavior-based tradition, extending it with slot assignment, near-slot braking, and overlap resolution mechanisms that improve stability during formation switches — a problem that Balch and Arkin's original work did not fully address.

2.3 LLM Integration in Robotics

The application of large language models to robotics is a recent and rapidly evolving research direction. Ahn et al. 's 2022 SayCan work demonstrated that a language model can evaluate which robot skills are feasible in a given situation, effectively grounding language in robot affordances. This introduced the idea that LLMs need not generate low-level motor commands — they can reason about what is possible and select from a library of available skills.

Liang et al.'s Code as Policies work in 2023 went further, using LLMs to generate executable code from natural language descriptions of robot tasks. This is powerful but risky for real-time control because generated code must be validated before execution.

The current system takes a more conservative and reliable approach. The LLM is asked only to select from five pre-defined strategic commands. It cannot generate new behaviors or modify the simulation code. This dramatically reduces the risk of unexpected outputs while still allowing natural language reasoning to influence mission priorities in a way that a pure rule engine cannot.

2.4 Spatial Indexing for Multi-Agent Systems

The problem of efficiently querying which agents are near a given agent in a large simulation has been studied extensively in the context of game engine design and scientific computing. Uniform grid partitioning, reviewed in Pharr, Jakob, and Humphreys' Physically Based Rendering textbook, divides the simulation space into fixed-size cells and registers each agent in its cell. Neighbor queries then inspect only the surrounding cells rather than the full agent list.

For uniform density distributions the uniform grid achieves O(1) neighbor query cost per agent, reducing total per-frame neighbor search cost to O(n). This is the approach implemented in the simulation's SpatialGrid.ts module. Alternative data structures such as k-d trees offer better worst-case guarantees under non-uniform distributions but carry higher maintenance cost under frequent agent movement, making the uniform grid the practical choice for a real-time simulation.

3. Algorithms and the Monitoring Brain

3.1 Boids Flocking Algorithm

The Boids algorithm is the computational core of how individual drones decide to move in relation to their neighbors. Each drone queries its local neighborhood and computes three steering forces that are blended into its final movement decision.

Rule	Computation	Effect on Drone
Separation	Repulsion force inversely proportional to distance from each neighbor within radius	Prevents crowding and physical collision between agents
Alignment	Average velocity vector of all neighbors within perception radius	Drone gradually matches the heading of its local group
Cohesion	Vector toward the average position of local neighbors	Drone stays loosely connected to the group rather than drifting away

The three forces are combined with tunable weights. Higher separation weight produces a more dispersed formation. Higher cohesion weight tightens the cluster. Alignment weight controls how quickly the group synchronizes direction after a disturbance. The leader command system adjusts these weights in response to mission priorities.

3.2 Steering Behaviors

On top of the Boids foundation, each drone runs four additional steering behaviors that handle specific navigation tasks.

Seek

Seek computes the desired velocity as a normalized vector from the drone's current position toward a target, scaled to maximum speed. The steering force is the difference between this desired velocity and the drone's current velocity. It is used when drones are far from their formation slots and need to move toward them directly.

```
desired = normalize(target - position) × maxSpeed
steering = desired - currentVelocity
```

Arrive

Arrive extends Seek by introducing a slowing radius. When the drone is outside this radius it behaves like Seek. Inside the radius, desired speed scales linearly down to zero at the target. This produces smooth, non-oscillating arrival at formation slots and is the mechanism behind the near-slot braking behavior.

```
if distance < slowingRadius:
   desiredSpeed = maxSpeed × (distance / slowingRadius)
```

Wander

Wander generates smooth random motion by maintaining a virtual circle projected ahead of the drone. A target point on the edge of this circle drifts by a small random angle each tick. The drone steers toward this drifting point, producing organic-looking exploration behavior. Scout drones have high wander weights. Worker drones have near-zero wander to stay on-station.

Obstacle Avoidance

Obstacle avoidance projects a detection box forward in the direction of travel. The length of the box scales with current speed — faster drones look further ahead. If any hazard geometry intersects the box, a lateral repulsion force is generated perpendicular to the travel direction, proportional to the inverse of the clearance distance.

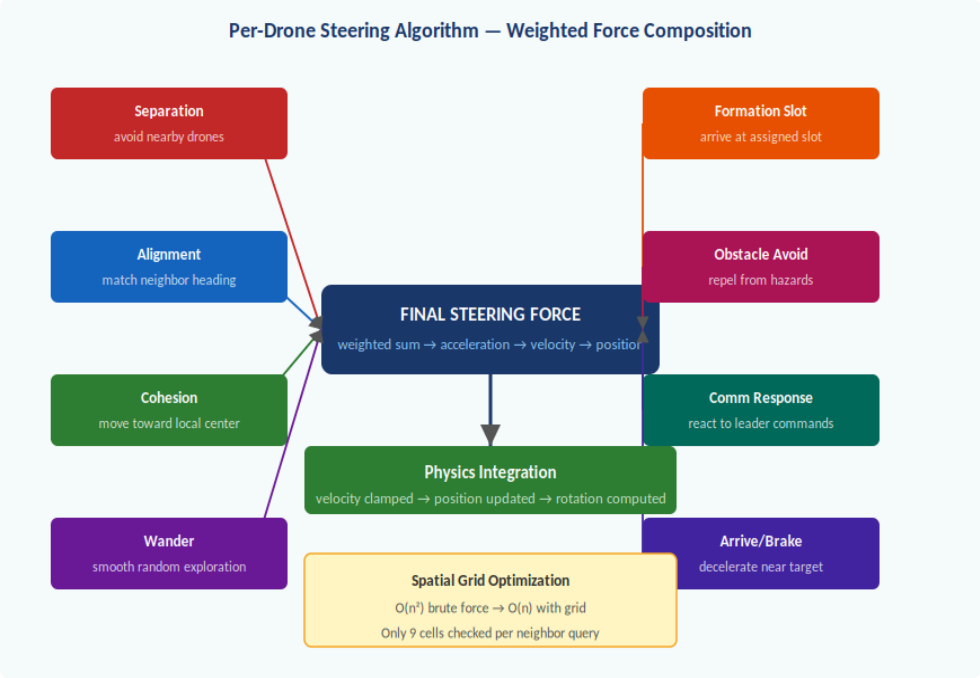

Figure 1. Per-drone steering force composition. Seven force sources are weighted and summed into a final acceleration vector each simulation tick.

3.3 Spatial Grid for Fast Neighbor Search

Without spatial indexing, every drone must compute its distance to every other drone every frame to find its neighbors. For a fleet of n drones this costs $O(n^2)$ distance evaluations per tick. At 50 drones running at 60 frames per second that is 147,000 evaluations per second — manageable but already limiting further scaling.

The spatial grid divides the canvas into cells of configurable size (default 120 pixels). Every tick, each drone registers itself in the cell containing its center position. A neighbor query for any drone then reads only the contents of the 9 cells surrounding that drone's cell, typically returning 10–20 candidates rather than all 50. After filtering these candidates by the actual perception radius, the drone has its true neighbor set.

Metric	Brute Force $O(n^2)$	Spatial Grid $O(n)$
Checks at n=32	992 per tick	~120 per tick
Checks at n=50	2,450 per tick	~180 per tick
Checks at n=100	9,900 per tick	~360 per tick
Frame rate (n=50)	43 fps (measured)	60 fps (measured)
Grid rebuild cost	—	One registration per drone per tick

3.4 Building the Monitoring Brain

The monitoring brain is the Leader Agent — the cognitive center of the swarm. It has two operational modes: an LLM-backed mode that queries Groq for strategic decisions, and a deterministic rule-based fallback mode that operates locally without any network dependency.

How the Brain Receives Information

Every simulation tick, all worker drones publish their current state to the shared message bus. The leader reads all messages and assembles a compressed telemetry snapshot containing the following fields:

- Active drone count and total fleet size
- Average energy level across all active drones
- Average health level across all active drones
- Count of HAZARD_DETECTED messages received this tick
- Count of near-collision events detected this tick
- Count of DISTRESS messages (critically low health drones)
- Current formation mode and active autopilot waypoint

Positions are rounded to the nearest integer. Velocities are reduced to scalar speed. This compression reduces the token cost of each LLM query by approximately 40% compared to sending raw state data.

LLM Brain Mode

In LLM mode, the compressed telemetry snapshot is sent to the Groq inference API with a strict system prompt instructing the model to respond with exactly one of five command words. The system prompt eliminates explanatory text, markdown formatting, and hedging — the model must output a single strategic command and nothing else.

Five Strategic Commands Available to the Brain

- HOLD_FORMATION — Maintain current formation. Increase slot attraction weight, reduce wander.
- REGROUP — Pull the swarm together. Increase cohesion and centroid pull.
- EXPAND_SEARCH — Widen coverage. Increase wander, reduce slot constraint for scouts.
- AVOID_HAZARDS — Prioritize obstacle clearance. Increase repulsion weights fleet-wide.
- CONSERVE_ENERGY — Damp all movement forces. Minimize wander and formation corrections.

Rule-Based Fallback Brain

When Groq is unavailable — due to rate limiting, network failure, or configuration — the local planner evaluates the same telemetry snapshot using a priority-ordered rule chain. This chain runs in microseconds and produces a decision synchronously within the same tick.

```
if distress_count > 0          → CONSERVE_ENERGY
elif avg_energy < 25%          → CONSERVE_ENERGY
elif hazard_count >= 2         → AVOID_HAZARDS
elif near_collision_count > N  → REGROUP
elif avg_health < 50%          → AVOID_HAZARDS
else                           → HOLD_FORMATION
```

A cooldown timer prevents retry storms after a rate-limit event. The UI shows clearly whether the brain is operating in LLM mode or fallback mode at all times.

4. Drone Communication — Peer to Peer and to Leader

4.1 The Message Bus Architecture

Drone communication in this system does not use direct peer-to-peer messaging. Instead, all drones communicate through a shared message bus — a central in-memory structure that any drone can write to and any drone can read from within a single simulation tick.

This design decision has significant consequences for system behavior. Because the bus is the single communication channel, adding a new drone type or a new message type never requires changing the code of existing drones. Drones are decoupled from each other by design. The leader does not need to know which specific drone sent a particular message — it only sees the aggregate picture.

Figure 2. Message bus communication architecture. Worker drones publish status upward; the leader reads, decides, and broadcasts commands back down. The bus is cleared and rebuilt every tick.

4.2 Worker-to-Leader Communication

Each worker drone publishes at minimum one STATUS_REPORT message per tick containing its current position, velocity, energy level, health level, and role profile. Drones that detect specific conditions publish additional messages:

Message Type	Trigger Condition	Data Carried	Leader Response
STATUS_REPORT	Every tick, always	Position, speed, energy, health, role	Builds swarm-wide telemetry picture
LOW_ENERGY	Energy drops below configured threshold	Drone ID, current energy level	May trigger CONSERVE_ENERGY command
DISTRESS	Health drops below critical threshold	Drone ID, current health level	Likely triggers CONSERVE_ENERGY

Message Type	Trigger Condition	Data Carried	Leader Response
HAZARD_DETECTED	Hazard enters perception radius	Hazard type, approximate location	May trigger AVOID_HAZARDS command

4.3 Leader-to-Worker Communication

After the leader has processed the tick's incoming messages and made a planning decision — either from the LLM or the local fallback — it publishes a LEADER_COMMAND message to the same bus. This message carries the selected strategic command.

Every drone reads the LEADER_COMMAND message during its steering force computation phase. The command maps to a weight adjustment table that modifies how strongly each steering force acts on that drone during this and subsequent ticks, until a new command arrives.

Command	Separation Weight	Cohesion Weight	Formation Weight	Wander Weight	Obstacle Weight
HOLD_FORMATION	Normal	Normal	High	Low	Normal
REGROUP	Normal	High	High	Low	Normal
EXPAND_SEARCH	Normal	Low	Low	High	Normal
AVOID_HAZARDS	High	Normal	Normal	Low	Very High
CONSERVE_ENERGY	Normal	Normal	Normal	Near 0	Normal

4.4 Internal Detection — How a Drone Knows Something Is Wrong

Each drone continuously monitors its own internal state every tick, independent of the message bus. This self-monitoring layer operates below the communication layer and is the primary source of LOW_ENERGY and DISTRESS messages.

- Energy monitoring: energy drains each tick proportional to movement speed and wander activity. Inside a magnetic field, energy drains faster. When energy crosses the low threshold, the drone publishes LOW_ENERGY.
- Health monitoring: health decrements when the drone is inside an electrical storm radius or when a direct collision occurs. When health crosses the critical threshold, DISTRESS is published.
- Collision detection: each drone checks its distance to all neighbors returned by the spatial grid query. Near-collision flags the drone as unstable and increases its local separation weight. Direct collision applies an impulse and damages health.
- Hazard detection: each drone checks its distance to every active hazard in the environment. If any hazard falls within the perception radius, a HAZARD_DETECTED message is published and the appropriate force effects are applied immediately.

5. Software, Hardware, and Technical Requirements

5.1 Software Stack

Component	Technology	Purpose
Frontend Framework	React 18 + TypeScript	Component-based UI, type-safe simulation logic
Build Tool	Vite	Fast development server, production bundling, API route handling
Rendering	HTML5 Canvas (2D Context)	High-performance frame-by-frame simulation rendering
Styling	CSS Modules / Tailwind	Dashboard panel layout and responsive design
AI Inference	Groq Cloud API	Low-latency LLM inference for strategic planning
LLM Model	llama-3.1-8b-instant	Small, fast model minimizing token usage and rate-limit risk
State Persistence	Browser localStorage	Mission state save and reload across sessions
Deployment	GitHub Pages (static)	Public demo deployment without backend server
Version Control	Git + GitHub Actions	CI/CD pipeline with TypeScript checks and build validation

5.2 Core Software Modules

The simulation is organized into eight TypeScript modules under the src/swarm/ directory. Each module has a single clearly defined responsibility.

Module	File	Responsibility
Simulation Engine	Simulation.ts	Main tick loop, physics integration, state management
Drone Agent	Drone.ts	Per-drone state, role profile, internal detection
Internal Agent	InternalAgent.ts	Steering force computation, weight table application
Leader Agent	LeaderAgent.ts	Telemetry aggregation, Groq query, fallback planning
Message Bus	MessageBus.ts	Publish-subscribe channel, per-tick clear and rebuild
Spatial Grid	SpatialGrid.ts	Cell registration, neighbor query, grid management

Module	File	Responsibility
Environment	Environment.ts	Hazard state, per-tick damage application, obstacle data
Canvas Renderer	CanvasRenderer.tsx	Frame rendering, drone visuals, hazard overlays, trails

5.3 Development Environment Requirements

- Node.js version 20 or newer — required for Vite and the npm ecosystem
- npm package manager — for dependency installation (npm ci for reproducible installs)
- A modern browser — Chrome 115+, Firefox 118+, or Safari 17+ for Canvas performance
- Groq API key (optional) — free tier available at console.groq.com, required only for LLM planning mode
- Environment file — a local .env file with GROQ_API_KEY and GROQ_MODEL variables for local development

5.4 Hardware Requirements

Because the simulation runs entirely in the browser with no native code, hardware requirements are defined by the browser's JavaScript engine and Canvas rendering performance rather than by the application directly.

Configuration	Minimum Spec	Recommended Spec
CPU	Dual-core 2.0 GHz	Quad-core 3.0 GHz or better
RAM	4 GB system RAM	8 GB system RAM
GPU	Integrated graphics	Dedicated GPU (improves Canvas compositing)
Display	1280 × 720 resolution	1920 × 1080 or higher
Network	Required for Groq mode	Low latency broadband for LLM planning
Drone count	Up to 32 (stable 60fps)	Up to 50 (stable 60fps with spatial grid)

5.5 API and Deployment Considerations

The Groq API key must never be exposed in browser JavaScript. The Vite development server's plugin system intercepts requests to /api/leader-plan and injects the key server-side. For production deployment with LLM support, the application must run on a platform that supports serverless functions or server-side routes.

- Vercel — serverless functions via api/ directory, easiest migration from Vite
- Netlify Functions — similar to Vercel, supports environment variable injection
- Cloudflare Workers — edge deployment with minimal cold-start latency
- GitHub Pages (current) — static only, LLM mode unavailable, local fallback active

6. Conclusion — Operational Conditions and Requirements

6.1 When This System Works Best

The swarm coordination system produces its most stable and useful behavior when a specific set of operational conditions are met. Understanding these conditions is important for anyone deploying the system for real monitoring experiments.

Condition	Optimal Range	Degraded Behavior Outside Range
Fleet size	8 to 50 drones	Below 8: formations lose shape. Above 50: frame rate drops without grid.
Formation spacing	40–120 px between slots	Too tight: overlap collisions. Too wide: formation loses cohesion.
Hazard density	1–3 active hazards	4+ hazards cause conflicting avoidance forces and instability.
Groq rate limit	≤1 query per 5 seconds	Higher frequency triggers rate limits; fallback activates automatically.
Canvas resolution	800×600 minimum	Smaller canvas causes boundary collisions and compression artifacts.
Browser tab focus	Tab must be active	Browsers throttle requestAnimationFrame in background tabs.

6.2 Situations Where the System Excels

Based on the practical scenario evaluations conducted during development, the system performs most effectively in the following monitoring contexts:

- Area coverage monitoring: EXPAND_SEARCH mode with Scout role drones distributes agents across the canvas efficiently, maximizing sensor coverage without explicit path planning
- Hazard-aware patrol: the combination of HAZARD_DETECTED messages and the AVOID_HAZARDS command creates reactive obstacle avoidance that adjusts in real time as hazards are placed or moved
- Formation-based inspection: Grid and Hexagon formations with narrow spacing allow the swarm to sweep a region in a structured, overlapping pattern that minimizes coverage gaps
- Endurance missions: CONSERVE_ENERGY mode significantly reduces energy drain rate, extending effective operating time for long-duration monitoring tasks
- Fault tolerance testing: the AI-to-fallback handoff scenario demonstrates resilience — even with Groq unavailable, the swarm continues operating safely and purposefully

6.3 Minimum Requirements to Deploy

1. A modern browser with stable 60fps Canvas performance on the target hardware
2. A swarm size between 8 and 50 drones depending on the monitoring area and coverage requirements
3. A formation mode selected for the terrain: Grid or Hexagon for structured sweeps, V-Shape or Leader for directional patrol, Circle for perimeter monitoring
4. At least one Scout drone in the fleet for hazard detection and area awareness
5. At least one Defender drone for formation stability in high-hazard environments
6. The Leader/Relay drone properly initialized as the communication hub
7. Groq API key configured if LLM-based planning is desired, or reliance on local fallback for offline operation
8. Autopilot waypoints configured if the monitoring mission covers a defined geographic route

6.4 Architecture Diagram

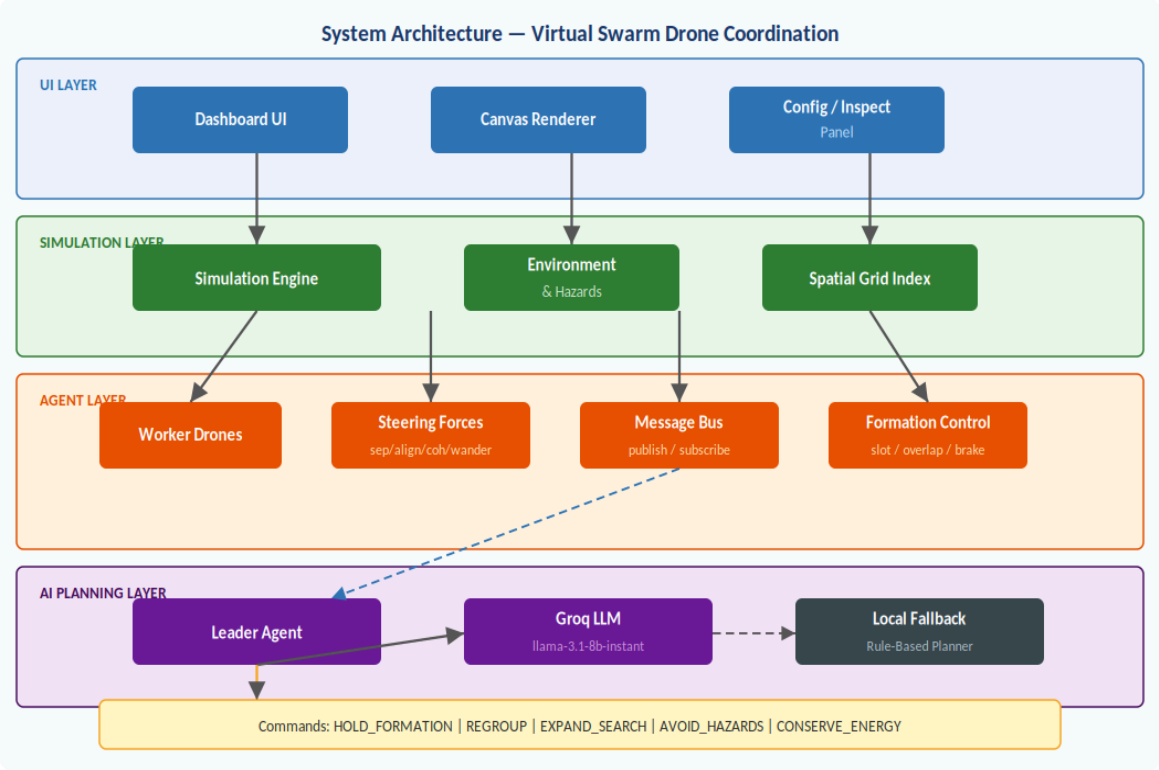

Figure 3. Complete system architecture from UI layer through simulation, agent, and AI planning layers. Data flows downward; telemetry and commands flow bidirectionally between the agent and AI layers.

6.5 Final Assessment

Virtual Swarm Drone Coordination demonstrates that a well-layered architecture can deliver genuinely useful autonomous swarm behavior within the constraints of a browser runtime and a free-tier AI inference API. The separation between local drone physics, peer communication, formation control, and AI-assisted mission planning is the key architectural decision that makes the system both robust and extensible.

The local fallback planner ensures that the system never depends on AI availability to remain safe and operational. The spatial grid index ensures that computational cost remains practical as fleet size grows. The message bus ensures that communication remains clean and decoupled as new drone types and message types are added. And the five-command strategic vocabulary provides enough expressiveness for meaningful mission differentiation while remaining simple enough for a small LLM to use reliably.

The result is a prototype that bridges the gap between simple flocking demos and real-world autonomous swarm platforms — one that any developer, researcher, or student can run in a browser, modify in TypeScript, and use as a foundation for exploring the coordination challenges that autonomous drone fleets will need to solve at scale.